Modeling with XGBoost

Elias Grant 306445678

2026-05-03

Libraries and Data

```r
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.4.2

## Warning: package 'ggplot2' was built under R version 4.4.3

## Warning: package 'tibble' was built under R version 4.4.3

## Warning: package 'tidyr' was built under R version 4.4.3

## Warning: package 'readr' was built under R version 4.4.3

## Warning: package 'purrr' was built under R version 4.4.3

## Warning: package 'dplyr' was built under R version 4.4.3

## Warning: package 'stringr' was built under R version 4.4.3

## Warning: package 'forcats' was built under R version 4.4.3

## Warning: package 'lubridate' was built under R version 4.4.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.4.3
```

```r
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.4.3

## Loading required package: lattice

## Warning: package 'lattice' was built under R version 4.4.3

##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
data <- read_rds("stuff_data.rds")
```

Feature Engineering

Adjust release location, approach angle, and horizontal break to account for pitcher handedness

```r
# Horizontal release location
data <- data |>
  mutate(RelSide2 = ifelse(PitcherThrows == "Left", -1 * RelSide, RelSide))

# Horizontal break
data <- data |>
  mutate(HorzBreak2 = ifelse(PitcherThrows == "Left", -1 * HorzBreak, HorzBreak))

# Spin induced horizontal break
data <- data |>
  mutate(pfxx2 = ifelse(PitcherThrows == "Left", -1 * pfxx, pfxx))

# Horizontal approach angle
data <- data |>
  mutate(HorzApprAngle2 = ifelse(PitcherThrows == "Left", -1 * HorzApprAngle, HorzApprAngle))
```

Create adjusted velocity and movement metrics relative to fastball

```r
# Velocity Differential
fb_velo <- data |>
  group_by(Pitcher) |>
```

```r
  filter(TaggedPitchType == "Fastball" | TaggedPitchType == "FourSeamFastBall") |>
  summarize(avg_fbvelo = mean(RelSpeed, na.rm = TRUE))

data <- data |>
  left_join(fb_velo, by = "Pitcher")

data <- data |>
  mutate(speed_diff = RelSpeed - avg_fbvelo)

# Movement Differential
fb_break <- data |>
  group_by(Pitcher) |>
  filter(TaggedPitchType == "Fastball" | TaggedPitchType == "FourSeamFastBall") |>
  summarize(avg_fbbreak = mean(pfxx2, na.rm = TRUE))

data <- data |>
  left_join(fb_break, by = "Pitcher")

data <- data |>
  mutate(break_diff = pfxx2 - avg_fbbreak)

# Remove excess tibbles
rm(fb_break, fb_velo)


# Export dataset
saveRDS(object = data, file = "stuffplus_data.rds")
```

Modeling XGBoost

```r
# Data Preparation
data <- data |>
  select(c("Pitcher", "TaggedPitchType", "RelSpeed", "SpinRate", "RelHeight",
           "RelSide2", "Extension", "InducedVertBreak", "pfxx2", "VertApprAngle",
           "HorzApprAngle2", "speed_diff", "break_diff", "linear_weight", "lw_event")) |>
  drop_na()

# Convert lw_event to a 0-indexed numeric factor required for XGBoost multi-class
data$lw_event <- as.factor(data$lw_event)
data$class_label <- as.numeric(data$lw_event) - 1
num_classes <- length(unique(data$class_label))

set.seed(10293939)
train_index <- createDataPartition(data$linear_weight, p = 0.8, list = FALSE)

train_df <- data[train_index,]
test_df <- data[-train_index,]

feature_cols <- c("RelSpeed", "SpinRate", "RelHeight", "RelSide2", "Extension",
                  "InducedVertBreak", "pfxx2", "VertApprAngle", "HorzApprAngle2",
                  "speed_diff", "break_diff")
```

```r
features1 <- as.matrix(train_df[, feature_cols])
features2 <- as.matrix(test_df[, feature_cols])


# REGRESSION MODEL
label1_reg <- train_df$linear_weight
label2_reg <- test_df$linear_weight

train_reg <- xgb.DMatrix(data = features1, label = label1_reg)
test_reg  <- xgb.DMatrix(data = features2, label = label2_reg)

tune_grid <- expand.grid(
  eta = c(0.01, 0.05, 0.1),
  max_depth = c(3, 5, 7),
  subsample = c(0.8, 1.0),
  colsample_bytree = c(0.8, 1.0)
)

best_rmse <- Inf
best_params_reg <- list()

cat("Starting Grid Search for Regression Model...\n")
```

```
## Starting Grid Search for Regression Model...
```

```r
for (i in 1:nrow(tune_grid)) {
  params <- list(
    objective = "reg:squarederror",
    eval_metric = "rmse",
    eta = tune_grid$eta[i],
    max_depth = tune_grid$max_depth[i],
    subsample = tune_grid$subsample[i],
    colsample_bytree = tune_grid$colsample_bytree[i]
  )

  set.seed(10293939)

  mdl <- xgb.train(
    params = params,
    data = train_reg,
    nrounds = 200,
    evals = list(val = test_reg),
    early_stopping_rounds = 10,
    verbose = 0
  )

  # Manually calculate RMSE
  preds <- predict(mdl, test_reg)
  current_rmse <- sqrt(mean((preds - label2_reg)^2))

  if (current_rmse < best_rmse) {
    best_rmse <- current_rmse
```

```r
    best_params_reg <- params
  }
}

cat("Best Regression RMSE:", best_rmse, "\n")
```

```
## Best Regression RMSE: 0.1217067
```

```r
print(best_params_reg)
```

```
## $objective
## [1] "reg:squarederror"
##
## $eval_metric
## [1] "rmse"
##
## $eta
## [1] 0.05
##
## $max_depth
## [1] 7
##
## $subsample
## [1] 0.8
##
## $colsample_bytree
## [1] 1
```

```r
# Train Final Regression Model
model_reg_final <- xgb.train(
  params = best_params_reg,
  data = train_reg,
  nrounds = 500,
  evals = list(train = train_reg, test = test_reg),
  early_stopping_rounds = 10,
  print_every_n = 50
)
```

```
## Multiple eval metrics are present. Will use test_rmse for early stopping.
## Will train until test_rmse hasn't improved in 10 rounds.
##
## [1]  train-rmse:0.122528 test-rmse:0.124151
## [51] train-rmse:0.116661 test-rmse:0.121955
## [101]    train-rmse:0.113168 test-rmse:0.121729
## Stopping. Best iteration:
## [119]    train-rmse:0.111922 test-rmse:0.121712
##
## [119]    train-rmse:0.111922 test-rmse:0.121712
```

```r
# CLASSIFICATION MODEL
```

```r
# Create DMatrix objects for classification
label1_class <- train_df$class_label
label2_class <- test_df$class_label

train_class <- xgb.DMatrix(data = features1, label = label1_class)
test_class  <- xgb.DMatrix(data = features2, label = label2_class)

# Reformat linear weights to a vector,
# Ensure run-value is constant for each classification
class_run_values <- train_df |>
  group_by(class_label) |>
  summarize(avg_rv = mean(linear_weight, na.rm = TRUE)) |>
  arrange(class_label) |>
  pull(avg_rv)

params_class <- list(
  objective = "multi:softprob",
  eval_metric = "mlogloss",
  num_class = num_classes,
  eta = 0.05,
  max_depth = 5
)

# Train Classification Model
model_class <- xgb.train(
  params = params_class,
  data = train_class,
  nrounds = 1000,
  evals = list(train = train_class, test = test_class),
  early_stopping_rounds = 10,
  print_every_n = 50
)
```

```
## Multiple eval metrics are present. Will use test_mlogloss for early stopping.
## Will train until test_mlogloss hasn't improved in 10 rounds.
##
## [1]  train-mlogloss:1.718761 test-mlogloss:1.732481
## [51] train-mlogloss:1.516929 test-mlogloss:1.566513
## [101]    train-mlogloss:1.439294 test-mlogloss:1.516557
## [151]    train-mlogloss:1.391996 test-mlogloss:1.492494
## [201]    train-mlogloss:1.351995 test-mlogloss:1.475466
## [251]    train-mlogloss:1.318461 test-mlogloss:1.462538
## [301]    train-mlogloss:1.288037 test-mlogloss:1.452253
## [351]    train-mlogloss:1.259903 test-mlogloss:1.444175
## [401]    train-mlogloss:1.232872 test-mlogloss:1.438100
## [451]    train-mlogloss:1.207846 test-mlogloss:1.433460
## [501]    train-mlogloss:1.183013 test-mlogloss:1.428657
## [551]    train-mlogloss:1.160741 test-mlogloss:1.425320
## [601]    train-mlogloss:1.138833 test-mlogloss:1.422382
## [651]    train-mlogloss:1.118924 test-mlogloss:1.420617
## [701]    train-mlogloss:1.097721 test-mlogloss:1.417166
## [751]    train-mlogloss:1.079231 test-mlogloss:1.415718
## Stopping. Best iteration:
```

```
## [756]    train-mlogloss:1.077301 test-mlogloss:1.415859
##
## [756]    train-mlogloss:1.077301 test-mlogloss:1.415859
```

```r
# Predictions and Evaluation

# Predict probabilities on test set
pred_probs <- predict(model_class, test_class)

# Calculate Expected Run Value (xRV) via matrix multiplication
test_df$predicted_xRV <- as.vector(pred_probs %*% class_run_values)

# Compare regression predictions vs classification xRV predictions
test_df$predicted_reg <- predict(model_reg_final, test_reg)

# View results
head(test_df[, c("linear_weight", "predicted_reg", "predicted_xRV")])
```

```
## # A tibble: 6 x 3
##   linear_weight predicted_reg predicted_xRV
##           <dbl>         <dbl>         <dbl>
## 1       -0.0541      -0.00587      -0.0267
## 2       -0.0702      -0.0200       -0.0251
## 3       -0.0461       0.00695       0.00718
## 4        0.0412      -0.0351       -0.0403
## 5       -0.0951      -0.00753      -0.0276
## 6       -0.0702      -0.0224       -0.0238
```

Feature Importance

```r
# Generate the importance matrices mapping your feature columns to the models
importance_reg <- xgb.importance(feature_names = feature_cols, model = model_reg_final)
importance_class <- xgb.importance(feature_names = feature_cols, model = model_class)

# Print the dataframes to the console
cat("\nTop Features: Regression Model\n")
```

```
##
## Top Features: Regression Model
```

```r
print(head(importance_reg, 5))
```

```
##             Feature       Gain      Cover  Frequency
##              <char>      <num>      <num>      <num>
## 1:    VertApprAngle 0.14980256 0.21434156 0.11100031
## 2:   HorzApprAngle2 0.13753961 0.18094835 0.10271556
## 3: InducedVertBreak 0.10005137 0.08839827 0.08553237
## 4:         RelSide2 0.09941706 0.06716286 0.10033753
## 5:        RelHeight 0.08869222 0.05714839 0.09680884
```

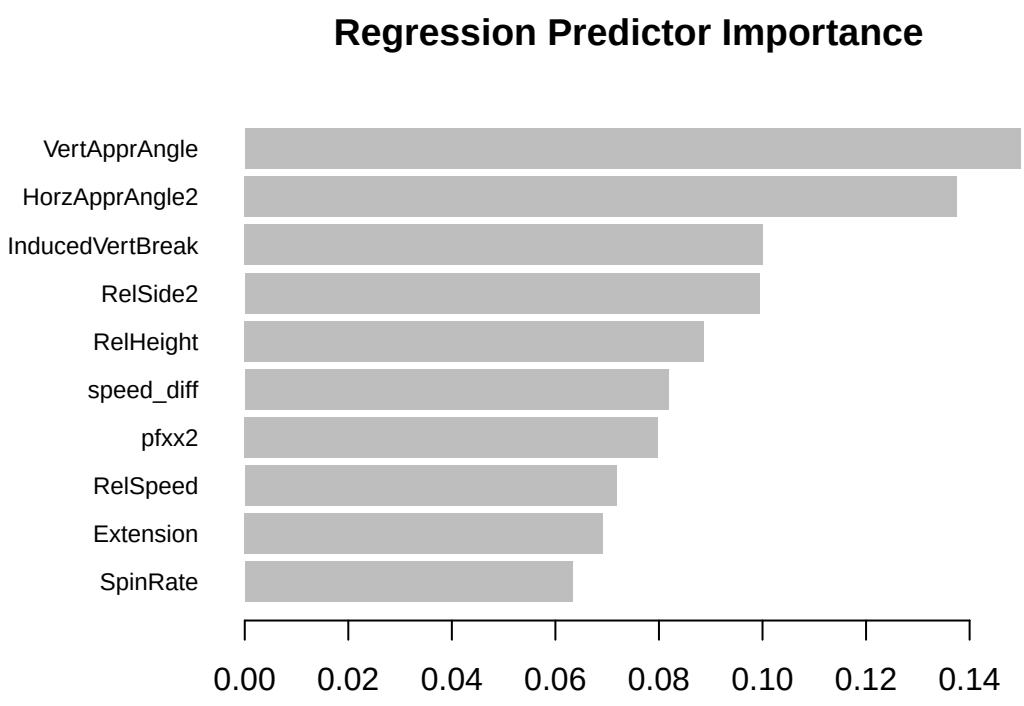
```r
cat("\nTop Features: Classification Model\n")
```

```
##
## Top Features: Classification Model
```

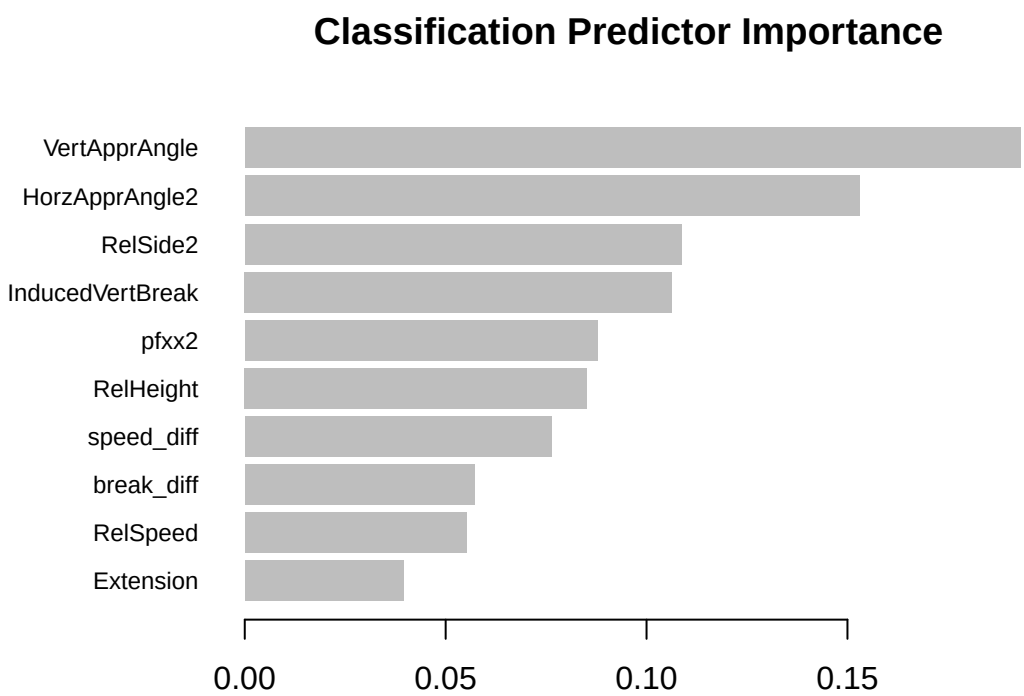
```r
print(head(importance_class, 5))
```

```
##             Feature       Gain      Cover  Frequency
##              <char>      <num>      <num>      <num>
## 1:    VertApprAngle 0.19303153 0.23473337 0.12236057
## 2:   HorzApprAngle2 0.15294542 0.17287842 0.11788585
## 3:         RelSide2 0.10868143 0.07275965 0.09835700
## 4: InducedVertBreak 0.10638239 0.10470004 0.09536352
## 5:            pfxx2 0.08782208 0.07856673 0.08829818
```

```r
# Plot the feature importance for visual comparison
xgb.plot.importance(
  importance_matrix = importance_reg,
  top_n = 10,
  measure = "Gain",
  main = "Regression Predictor Importance"
)
```



```r
xgb.plot.importance(
  importance_matrix = importance_class,
  top_n = 10,
  measure = "Gain",
  main = "Classification Predictor Importance"
)
```



Stuff+ Normalization

```r
# Calculate Stuff+ and group by pitcher

# Calculate league averages and standard deviations from the test set
mean_xRV <- mean(test_df$predicted_xRV, na.rm = TRUE)
sd_xRV <- sd(test_df$predicted_xRV, na.rm = TRUE)

mean_reg <- mean(test_df$predicted_reg, na.rm = TRUE)
sd_reg <- sd(test_df$predicted_reg, na.rm = TRUE)

# Score every individual pitch on the 100 scale
# Formula: 100 - (Z-score * 10).
# Note: subtract the z-score because a lower run value denotes better pitcher performance.
test_df <- test_df |>
  mutate(
```

```r
    pitch_StuffPlus_xRV = 100 - ((predicted_xRV - mean_xRV) / sd_xRV) * 10,
    pitch_StuffPlus_Reg = 100 - ((predicted_reg - mean_reg) / sd_reg) * 10
  )

# Group by Pitcher and summarize
pitcher_leaderboard <- test_df |>
  group_by(Pitcher) |>
  summarize(
    Pitches = n(),
    # Calculate the average Stuff+ score for the pitcher
    StuffPlus_xRV = round(mean(pitch_StuffPlus_xRV, na.rm = TRUE), 1),
    StuffPlus_Reg = round(mean(pitch_StuffPlus_Reg, na.rm = TRUE), 1)
  ) |>
  # Filter out low-sample pitchers
  filter(Pitches >= 20) |>
  # Sort by the Classification xRV model's Stuff+ descending
  arrange(desc(StuffPlus_xRV))

# View the top 10 pitchers by Stuff+
cat("\n--- Top 10 Pitchers by Stuff+ (xRV Model) ---\n")
```

```
##
## --- Top 10 Pitchers by Stuff+ (xRV Model) ---
```

```r
print(head(pitcher_leaderboard, 10))
```

```
## # A tibble: 10 x 4
##    Pitcher           Pitches StuffPlus_xRV StuffPlus_Reg
##    <chr>               <int>         <dbl>         <dbl>
##  1 Yesavage, Trey         21          110.          107
##  2 Hawkins, Dakota        24          105.          105.
##  3 Harrison, Charles      86          104.          104.
##  4 Omlid, Hunter          20          104.          103.
##  5 May, Aiden             43          104.          105.
##  6 Ruff, Rashad          158          104.          105.
##  7 Dixon, Joey            25          103.          104.
##  8 Wietgrefe, Connor      34          103.          105.
##  9 Hyde, Jace             25          103.          105.
## 10 Taylor, Grant          32          103.          104.
```

```r
# View the bottom 10 pitchers by Stuff+
cat("\n--- Bottom 10 Pitchers by Stuff+ (xRV Model) ---\n")
```

```
##
## --- Bottom 10 Pitchers by Stuff+ (xRV Model) ---
```

```r
print(tail(pitcher_leaderboard, 10))
```

```
## # A tibble: 10 x 4
##    Pitcher              Pitches StuffPlus_xRV StuffPlus_Reg
##    <chr>                  <int>         <dbl>         <dbl>
```

```
##  1 Ramos, Sachem            46          97           95.5
##  2 Thorsteinson, Justin     28          96.9         95.3
##  3 Christy, Bobby           21          96.6         95.4
##  4 Mathews, Quinn           20          96.6         96.8
##  5 Randall, Cal             21          96.5         99.7
##  6 Slayden, Zachary         44          96.3         95.5
##  7 Cherry, Grant            33          95.7         95.6
##  8 Mishoulam, Aaron         25          95.6         96.8
##  9 Sanchez, Frankie         45          95.6         94.5
## 10 Mann, Brandon            46          93.9         94.7
```

Overall, after cross-checking with Baseball Reference for the actual results of the top and bottom 10 pitchers, initial results are promising. Higher sample size is a must-have, but the work so far is a solid foundation to build on